Day 135 — Line-by-Line Syntax Explanation

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 20, Day 3 reference

Watch: Ali Ghodsi, Deep Learning, GAN, Generative adversarial networks, AAE, Fall 2023, Lecture 16 | GAN Challenges Explained: Mode Collapse, Instability & WGAN Solutions (CodeLucky)

`day135_gan_instability.py`, segment by segment, in the order `main()` uses things. The ring data set, the mode count (mode_stats), the watched training loop (train_ring) and the experiments are new; the GAN itself is Day 134's gan_step, generate and g_loss, and the networks are Day 126's init_net and Adam, all imported unchanged. Code boxes are cut from the real script by program (inline comments removed). Purple **MATH** boxes come before code that implements a formula. A glossary is at the end.

MATH BOX LEGEND

Symbols: k = number of modes · C = the true centres · F = generated samples · good = within 3 spreads of a centre · track = what was seen every 250 steps

Segment 0 — Setup

Imports

CODE

```
HERE = os.path.dirname(os.path.abspath(__file__))
sys.path.insert(0, HERE)
sys.path.insert(0, os.path.join(HERE, "..", "week-19-unsupervised-learning-ii"))
with contextlib.redirect_stdout(io.StringIO()):
    import day126_exercise as d126
    import day134_exercise as d134
```

- `sys.path.insert(0, HERE)` — this week's own folder, so Day 134's exercise file can be imported.
- `import day134_exercise as d134` — yesterday's parts, unchanged: today only uses them.

Segment 1 — The data and the measurements

demo_hand: the two gradients

MATH

$$d/dl\ [-\log \text{sigmoid}(l)] = D - 1, \quad d/dl\ [\log(1 - \text{sigmoid}(l))] = -D, \quad D = \text{sigmoid}(l)$$

CODE

```
for p in (0.5, 0.1, 0.01, 0.001):
    lf = np.full((1, 1), np.log(p / (1 - p)), np.float32)
    a, b = float(d134.g_loss(lf)[1][0, 0]), float(d134.g_loss(lf, "minimax")[1][0, 0])
    rows.append((p, a, b)); print(f"   {p:7.3f}   {a:+21.4f}   {b:+18.4f}   {a / b:5.0f}")
```

- `np.log(p / (1 - p))` — the logit whose sigmoid is p: the inverse of the sigmoid.
- `d134.g_loss(lf)[1][0, 0]` — g_loss returns (loss, gradient); [1] is the gradient array, [0, 0] its one entry.
- `f"{a:+21.4f}"` — a format with a sign always shown (+), 21 characters wide, 4 decimals.

ring and mode_stats

MATH

$$good(x) = [\min_c |x - C_c| < 3\sigma]; \quad \text{for a 2-D Gaussian, } P(good) = 1 - e^{-9/2} = 0.9889$$

CODE

```
def ring(n, rng, k=8, radius=2.0, sd=0.05):
    a = 2 * np.pi * rng.randint(0, k, n) / k
    return (np.c_[np.cos(a), np.sin(a)] * radius + sd * rng.standard_normal((n,
        2))).astype(np.float32)
def mode_stats(F, k=8, radius=2.0, sd=0.05):
    C = np.c_[np.cos(2 * np.pi * np.arange(k) / k), np.sin(2 * np.pi * np.arange(k) / k)] * radius
    D = np.sqrt(((F[:, None, :] - C[None]) ** 2).sum(-1)); near = D.argmin(1)
    good = D[np.arange(len(F)), near] < 3 * sd
    counts = np.bincount(near[good], minlength=k)
    return int((counts >= 0.01 * len(F)).sum()), float(good.mean()), counts
```

- `2 * np.pi * rng.randint(0, k, n) / k` — a random mode for every sample, as an angle around the circle.
- `np.c_[np.cos(a), np.sin(a)]` — np.c_ stacks the two columns side by side: points on the unit circle.
- `F[:, None, :] - C[None]` — broadcasting: (n, 1, 2) minus (1, k, 2) gives every sample's offset from every centre, (n, k, 2).
- `D[np.arange(len(F)), near]` — fancy indexing: each sample's distance to its own nearest centre.
- `np.bincount(near[good], minlength=k)` — good samples counted per mode, with a 0 for any mode never reached.

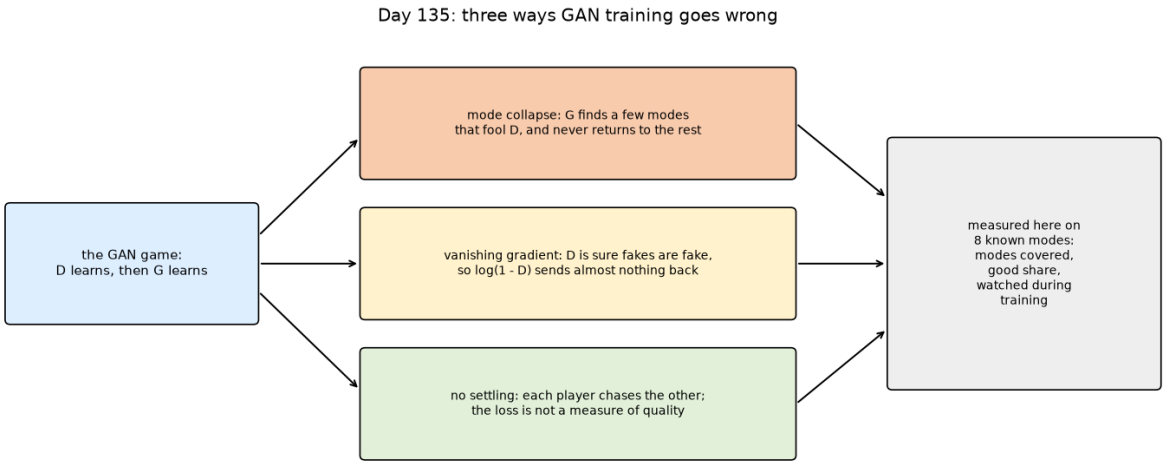
train_ring: a GAN watched every 250 steps

CODE

```
def train_ring(seed, steps=6000, kind="non-saturating", lr_g=1e-3, lr_d=1e-3, d_steps=1,
    every=250, k=8, z=2, head=0):
    rng = np.random.RandomState(seed); G = d126.init_net([z, 128, 128, 2], seed); D =
        d126.init_net([2, 128, 128, 1], seed + 100)
    oG, oD = d126.Adam(G, lr_g), d126.Adam(D, lr_d); track = []
    ev = np.random.RandomState(999).standard_normal((2000, z)).astype(np.float32)
    for t in range(1, steps + 1):
        oG.lr = 0.0 if t <= head else lr_g
        _, _, dr, df = d134.gan_step(G, D, oG, oD, ring(256, rng, k), rng, z, out="linear",
            kind=kind, d_steps=d_steps)
        if t % every == 0:
            m, g, c = mode_stats(d134.generate(G, ev, "linear")[0], k); track.append((t, m, g, c,
                dr, df))
    return G, track
```

- `oG.lr = 0.0 if t <= head else lr_g` — a head start: the generator's learning rate is 0 at first, so only the discriminator learns.
- `ev = ... standard_normal((2000, z))` — the same 2,000 noise vectors at every look, so changes come from the generator, not the noise.
- `if t % every == 0:` — look during training: every 250th step, not only after the last.

SYSTEM DIAGRAM



Three ways GAN training goes wrong, and what is measured here.

Segment 2 — The experiments

lost_and_found and run_seeds

CODE

```
def lost_and_found(track):
    have = [set(np.flatnonzero(c >= 20)) for _, _, _, c, _, _ in track]
    lost = sum(len(a - b) for a, b in zip(have, have[1:])); found = sum(len(b - a) for a, b in
        zip(have, have[1:]))
    return lost, found
def run_seeds(name, seeds=(0, 1, 2, 3, 4), **kw):
    marks = (500, 1000, 2000, 3000, 4000, 5000, 6000); out = {}; t0 = time.perf_counter()
    print(f"{name}: modes covered (of 8) at each step, then good share and D(fake) at the end")
    print("   seed  " + "".join(f"{m:>6,}" for m in marks) + "   good  D(fake)  lost  found  final
        counts per mode (of 2,000)")
    for s in seeds:
        G, tr = train_ring(s, **kw); at = {t: m for t, m, _, _, _, _ in tr}; lost, found =
            lost_and_found(tr)
        out[s] = (G, tr); print(f"   {s:4d}  " + "".join(f"{at[m]:6d}" for m in marks) + f"
            {tr[-1][2]:.2f}  {tr[-1][5]:7.2f}  {lost:4d}  {found:5d}  "
                        + " ".join(f"{c:3d}" for c in tr[-1][3]))
    print(f"   all 8 modes at the end in {sum(tr[-1][1] == 8 for _, tr in out.values())} of
        {len(seeds)} runs ({time.perf_counter() - t0:.0f} s)")
    return out
```

- `set(np.flatnonzero(c >= 20))` — the modes with at least 20 good samples, as a Python set.
- `len(a - b)` — set difference: modes covered at one look and gone at the next.
- `zip(have, have[1:])` — pairs of consecutive looks.
- `**kw` — collects any keyword arguments (kind, head, lr_g...) and passes them on to train_ring unchanged.
- `G, tr = train_ring(s, **kw)` — unpack the returned pair: the generator and its track.

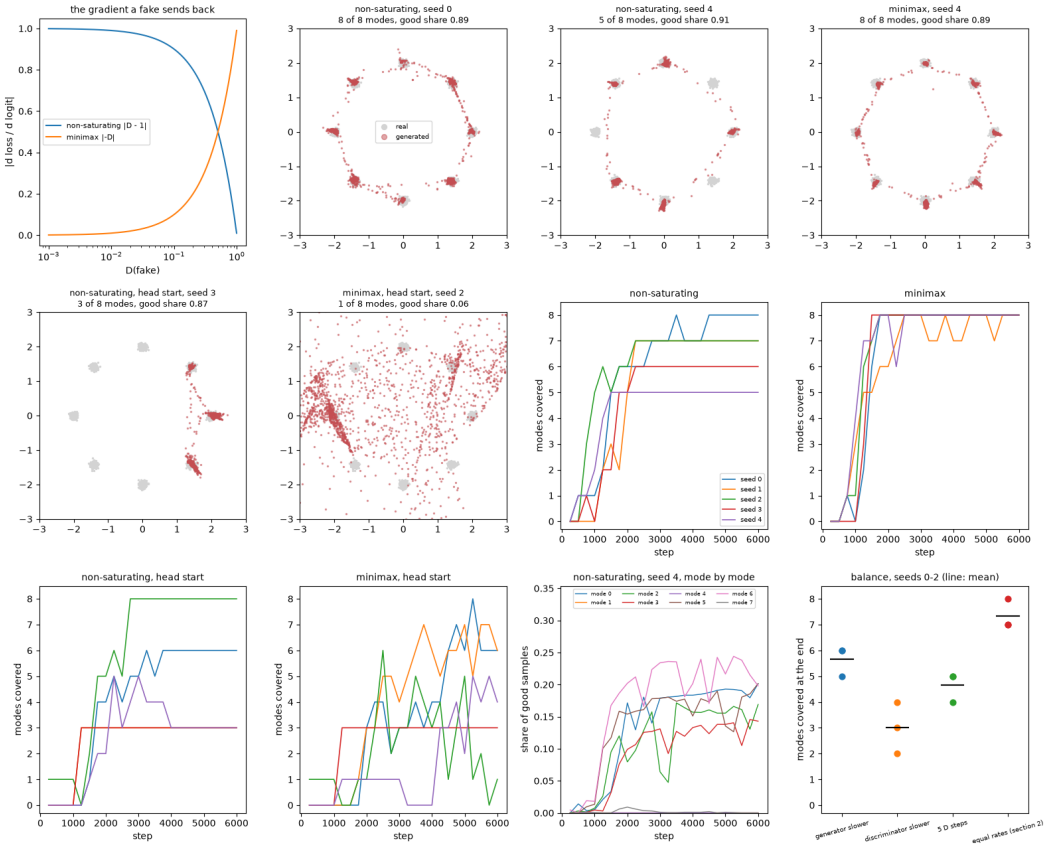
demo_balance: three settings, one loop

CODE

```
for name, kw in (("generator slower: G 0.0001, D 0.001", dict(lr_g=1e-4)), ("discriminator slower:
    G 0.001, D 0.0001", dict(lr_d=1e-4)),
                 ("5 discriminator steps per generator step", dict(d_steps=5))):
    out[name] = run_seeds(name, seeds=(0, 1, 2), **kw)
```

- `dict(lr_g=1e-4)` — a dictionary of the one setting that changes; everything else keeps train_ring's defaults.
- `run_seeds(name, seeds=(0, 1, 2), **kw)` — ** unpacks the dictionary back into keyword arguments.

REAL GRAPH



Real graph from the script.

Quick Syntax Glossary — new or reinforced constructs

- `np.c_[a, b]` — stack 1-D arrays as columns.
- `A[:, None, :] - B[None]` — broadcast every row of A against every row of B.
- `set(...); a - b` — a set, and the elements of a not in b.
- `f(**kw)` — pass a dictionary's items as keyword arguments.
- `min(R, key=...)` — the item with the smallest key, here the seed with the fewest modes.

End of Day 135 syntax reference. For the experiments and what they showed, see the Day 135 complete guide.